Deep learning for Bitcoin address classification: week 2

Nov 3, 2026 · @Ren

Summary

On addresses from entities the model has never seen, XGBoost with neighbour features is best (accuracy 0.945, macro F1 0.821), ahead of GraphSAGE (0.936 and 0.691 over three seeds): the graph information helps, deep learning adds nothing beyond it.

- **A transaction graph was built from the raw BABD data:** 13,093,228 transactions in 6,000 blocks, linking 94,851 labelled addresses by 287,596 payment and co-spend edges.
- **BABD-13's labels follow H1 clusters:** 99.3% of multi-address co-spend clusters carry a single label, and 1,414 addresses have contradictory labels, mostly blackmail against tumbler or individual wallet.
- **A random split overstates every model** by 4 to 11 accuracy points, because addresses of one entity end up on both sides; an entity split (whole co-spend clusters in training or test) removes that.
- **Venom link:** only 2 of the 4,382 Venom addresses are labelled in BABD, but 91 counterparties of the 8 Venom sets are, almost all exchanges, and set 25's address is co-spent with exchange addresses in 119 transactions.

Data: the BABD transaction window

The 22.4 GB of JSON in the BABD archive are not ready-made graphs: they are raw explorer API pages (BTC.com format) covering every transaction of blocks 600,000 to 605,999, around October and November 2019.

- **Layout:** one folder per block, one file per page of 50 transactions; 264,841 files. File names are offset from page numbers (`39_1.json` holds page 10), but a check on block 600,000 found all 39 pages and all 1,925 transactions.
- **Content per transaction:** inputs with the spent output's address and value, outputs with address and value, plus fee, size and block height, the same shape as the ColdCard blocks.
- **Sample check before the full scan:** 100 blocks (one in 60) already contained 7,120 BABD-labelled addresses, 69% of them exchanges.

- **Full scan:** one pass over all files in 10.6 minutes read 13,093,228 transactions, kept the 358,784 (2.7%) that touch a labelled address, and recorded each address's activity and the links between labelled addresses.
- **Coverage:** 94,966 of BABD's 544,462 labelled addresses (17%) appear in the window. BABD's features were computed over blocks 585,000 to 685,000, so most labelled addresses are simply inactive in these 6,000 blocks.

Label quality in BABD-13

BABD-13 contains contradictory labels for 1,414 addresses, and its labels are, in practice, assigned per co-spend (H1) cluster.

Duplicated and conflicting addresses

- 1,706 addresses appear more than once (2,094 extra rows); 1,414 of them carry two or more different labels.
- 1,008 extra rows have the same address and identical features but a different label, so no model can classify them correctly; 528 extra rows are exact copies.
- The conflicts almost all involve blackmail, the class the BABD paper builds mostly from weak user reports:

Labels found for one address (ten most frequent combinations)	Addresses
Blackmail + tumbler	790
Blackmail, twice (consistent)	292
Blackmail + individual wallet	232
Blackmail twice + tumbler	131
Blackmail + tumbler + individual wallet	94
Blackmail twice + individual wallet	63
Blackmail twice + tumbler + individual wallet	48
Government blacklist + money laundering	12
Blackmail + government blacklist	11
Blackmail + exchange	9

This explains part of week 1's weak results for blackmail (F1 0.59) and individual wallet (0.07). The 528 exact copies may have fallen on both sides of week 1's random split; they are 0.1% of the data, so the overall scores barely move, but the blackmail class is slightly flattered. For week 2, the 1,414 conflicting addresses were removed and 540,954 addresses with a single label were kept.

Labels follow H1 clusters

Among the 764 co-spend clusters with two or more addresses in the window, **99.3% carry a single label**. The BABD paper takes most labels from WalletExplorer, which labels whole clusters of addresses spent together, so co-spending addresses share a label largely by construction. The largest cluster holds 37,178 addresses (39.2% of the window), 37,172 of them labelled exchange. Any model that sees co-spend neighbours of a test address in training can copy the label from the cluster, which is why the entity split below is needed.

Graph and splits

The graph links 94,851 labelled addresses (each with its 148 BABD features) by 287,596 undirected edges, and 98% of its edges join two addresses of the same class.

- **Edges:** two labelled addresses are linked when they appear in the same transaction, either one paying the other (25,473 payment links) or both spent together (267,942 co-spend links). Large consolidations were linked as a star on the first input to avoid millions of pairs.
- **Connectivity:** 24.7% of addresses are isolated; the largest component holds 68,991 addresses (73%), the rest are small islands (152, 125, 51 addresses and so on).
- **Homophily:** 98.0% of edges join same-class addresses (co-spend 98.5%, payment 93.5%).

Class	Addresses in window	Isolated	In largest component	In a co-spend cluster of 2+
Centralized exchange	61,483	23.2%	76.4%	76.5%
Gambling	15,665	43.0%	50.8%	51.9%
Cyber-security service	13,722	9.7%	87.3%	85.7%
Tumbler	1,244	2.9%	96.2%	96.5%
P2P financial service	1,163	42.5%	57.5%	57.4%
Darknet market	980	27.2%	3.3%	72.8%

Class	Addresses in window	Isolated	In largest component	In a co-spend cluster of 2+
Blackmail	404	67.6%	25.7%	13.9%
P2P financial infrastructure	65	20.0%	0.0%	78.5%
Mining pool	65	4.6%	87.7%	58.5%
Individual wallet	56	53.6%	42.9%	25.0%

Darknet markets form their own small islands (only 3.3% in the largest component), while blackmail addresses are mostly isolated single addresses, consistent with labels from individual reports.

Two splits on the same 94,847 addresses (10 classes with at least 20 addresses; government blacklist and Ponzi dropped):

- **Entity split:** whole co-spend clusters go to training or test (`StratifiedGroupKFold`, 5 folds); the giant exchange cluster is kept in training. 82,269 training and 12,578 test addresses, with 0 clusters shared, so every test address belongs to an entity never seen in training.
- **Random split:** stratified, with the same test size (12,578).

Class	Test addresses, entity split	Test addresses, random split
Centralized exchange	5,166	8,153
Gambling	4,337	2,077
Cyber-security service	2,555	1,820
Darknet market	196	130
P2P financial service	167	154
Blackmail	81	54
Tumbler	47	165
Mining pool	13	9
Individual wallet	12	7
P2P financial infrastructure	4	9

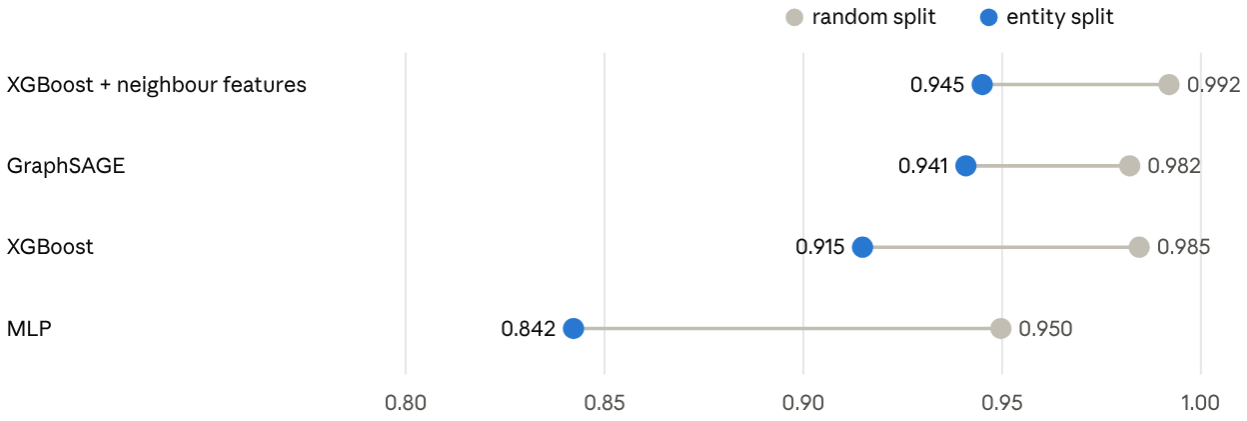
Only payment links cross from training to test in the entity split, and they carry features, never labels.

Results

On unseen entities, neighbour information is what matters: it lifts XGBoost from 0.915 to 0.945 accuracy and the neural model from 0.843 (MLP) to 0.936 (GraphSAGE), and XGBoost with neighbour features stays best on every metric.

On unseen entities every model loses 4 to 11 accuracy points

Test accuracy, BABD-13 addresses in the 6,000-block window, random split against entity split



Axis starts at 0.80. GraphSAGE and MLP: seed 42 (three-seed means on the entity split: 0.936 and 0.843).

Notebook output, 3 November 2026 · 12,578 test addresses per split

Models (same features, same splits): XGBoost (300 trees, depth 8) on the 148 features; XGBoost on the 148 features plus the mean of each address's neighbours' (log) features and its degree (297 features); an MLP and a two-layer GraphSAGE (128 units, dropout 0.2, square-root class weights, 300 full-batch epochs).

Model	Macro F1, entity	Weighted F1, entity	Macro F1, random	Weighted F1, random
XGBoost + neighbour features	**0.821**	**0.944**	**0.857**	**0.992**
GraphSAGE (3 seeds)	0.691 ± 0.046	0.939 ± 0.003	0.804 (1 seed)	0.983 (1 seed)
XGBoost	0.747	0.915	0.817	0.984
MLP (3 seeds)	0.593 ± 0.014	0.851 ± 0.002	0.699 (1 seed)	0.951 (1 seed)

Per class, entity split (F1; GraphSAGE and MLP with seed 42):

Class (test addresses)	XGBoost	MLP	GraphSAGE
Centralized exchange (5,166)	0.961	0.930	0.963

Class (test addresses)	XGBoost	MLP	GraphSAGE
Gambling (4,337)	0.920	0.859	0.959
Cyber-security service (2,555)	0.829	0.700	0.917
Darknet market (196)	0.970	0.906	0.933
P2P financial service (167)	0.988	0.889	0.979
Blackmail (81)	0.424	0.267	0.495
Tumbler (47)	0.688	0.354	0.357
Mining pool (13)	0.833	0.417	0.423
Individual wallet (12)	0.000	0.105	0.000
P2P financial infrastructure (4)	0.857	0.545	0.600

- **GraphSAGE beats plain XGBoost on the large classes** (cyber-security, gambling, blackmail) and loses on the small ones (tumbler, mining pool, P2P infrastructure, with 47, 13 and 4 test addresses), hence its higher accuracy but lower macro F1.
- **GraphSAGE's macro F1 is unstable across seeds** (0.66 to 0.74), because a few test addresses in the tiny classes decide it; its accuracy varies by only ±0.004.
- **Darknet market generalizes well to new markets** for every model (F1 0.91 to 0.97 on unseen entities).

Venom application

BABD labels say almost nothing about Venom addresses directly, but they identify exchange addresses among the counterparties of the Venom sets, and one set's onion-site address is very likely inside an exchange's wallet.

- **Direct overlap:** 2 of the 4,382 Venom addresses are labelled in BABD-13 (one blackmail, one cyber-security service), and none of the 8 sets studied in steps 3 and 4. *Hypothesis:* different periods (BABD 2019–2021, Venom April 2024) and different sources (WalletExplorer service clusters against payment addresses shown on onion pages).
- **A model-based application was not done:** a BABD-trained model needs BABD's 148 features, which cannot be reproduced for Venom addresses without reimplementing the BABD feature code, and proxy features would compare a 2019 window with 2016–2026 histories.
- **Counterparties:** 91 of the 58,544 counterparties of the 8 sets (0.16%) are labelled in BABD: 85 exchanges, 4 blackmail, 2 cyber-security. Sets 43, 45 and 667 have none.

Set	Labelled counterparties	Transactions with an exchange-labelled address
25	76 exchange, 2 blackmail	358 of 2,315
157	5 exchange, 1 blackmail	5
104	2 exchange, 1 blackmail, 1 cyber-security	2
209	1 exchange, 1 cyber-security	1
149	1 exchange	1

Set 25 and exchanges. In set 25's transactions the exchange addresses are almost never direct payment partners: the set pays an exchange in 3 transactions, and no exchange pays the set. Instead, the set's address is **spent together with exchange addresses in 119 transactions** (about 37% of the 319 transactions that spend it) and receives in the same transaction as an exchange address in 237. Under the common-input heuristic, the same wallet controls co-spent addresses, so the address shown on set 25's onion sites is most likely an address inside an exchange's wallet, such as a deposit address the exchange sweeps with others. *Hypothesis, well supported:* exchange consolidations are not CoinJoins, but the exchange labels date from 2019–2021 while set 25 is active from 2018 to 2026. This matches the cash-out pattern described by Lee et al. (2019).

Limitations

- **One window of 6,000 blocks (late 2019):** only 17% of BABD's labelled addresses are active in it, and the graph only links labelled addresses, so links through unlabelled addresses are lost.
- **Labels:** BABD labels follow H1 clusters; 1,414 conflicting addresses were removed, but remaining labels inherit WalletExplorer's clustering errors.
- **Entity split by co-spend cluster:** an approximation of entities; the giant exchange cluster (39% of addresses) is always in training, and four classes have 4 to 47 test addresses, so macro F1 is noisy.
- **Neural models barely tuned:** one architecture, fixed learning rate, 300 epochs; three seeds on the entity split only.
- **Venom:** the counterparty analysis rests on 91 labelled addresses and 2019–2021 labels, and the set-25 conclusion relies on the common-input heuristic.

Appendix: code

Cells in the order they ran (paths relative to the notebook; data under `data/babd/`, git-ignored).

B.1 The archive and the label table

```
import zipfile, json, collections, time
import numpy as np, pandas as pd
from collections import Counter, defaultdict

BABD = r"C:\path\to\BABD.zip"
babd = pd.read_csv(r"data/babd/BABD-13.csv")
babd_lab = babd[["account", "label"]]
names = ["Blackmail", "Cyber-security service", "Darknet market",
"Centralized exchange",
         "P2P financial infrastructure", "P2P financial service", "Gambling",
         "Government criminal blacklist", "Money laundering", "Ponzi scheme",
         "Mining pool", "Tumbler", "Individual wallet"]

with zipfile.ZipFile(BABD) as z:
    jsons = [i.filename for i in z.infolist() if
i.filename.endswith(".json")]
by_block = defaultdict(list)
for f in jsons:
    by_block[f.split("/")[2]].append(f)          # folder name = block height
```

B.2 Full scan of the 6,000 blocks

```
labels = dict(zip(babd_lab["account"], babd_lab["label"]))
edges = Counter()
act = defaultdict(lambda: [0, 0, 0, None, None])    # n_tx, sent, received,
first block, last block
n_tx_total = n_tx_kept = 0
with zipfile.ZipFile(BABD) as z:
    for b in sorted(by_block, key=int):
        h = int(b)
        for f in by_block[b]:
            for tx in json.load(z.open(f))["data"]["list"]:
                n_tx_total += 1
                ins = [(a, i.get("prev_value") or 0) for i in tx["inputs"]
for a in (i.get("prev_addresses") or [])]
                outs = [(a, o.get("value") or 0) for o in tx["outputs"] for a
in (o.get("addresses") or [])]
                lab_in = list(dict.fromkeys(a for a, _ in ins if a in
labels))
                lab_out = list(dict.fromkeys(a for a, _ in outs if a in
```

```
labels))
                if not lab_in and not lab_out:
                    continue
                n_tx_kept += 1
                for a in set(lab_in) | set(lab_out):
                    s = act[a]; s[0] += 1
                    s[3] = h if s[3] is None else s[3]; s[4] = h
                for a, v in ins:
                    if a in labels: act[a][1] += v
                for a, v in outs:
                    if a in labels: act[a][2] += v
                for a in lab_in:
                    for c in lab_out:
                        if a != c: edges[(a, c, "pay")] += 1
                for a in lab_in[1:]:
                    edges[(lab_in[0], a, "cospend")] += 1
act_df = pd.DataFrame.from_dict(act, orient="index", columns=["n_tx", "sent",
"received", "first_block", "last_block"])
edges_df = pd.DataFrame([(a, c, k, w) for (a, c, k), w in edges.items()],
columns=["src", "dst", "kind", "n_tx"])
```

B.3 Conflicting labels, graph, co-spend clusters

```
import networkx as nx

per_addr = babd_lab.groupby("account")["label"].agg(["nunique", "first"])
lab_clean = per_addr.loc[per_addr["nunique"] == 1, "first"]
nodes = act_df.index[act_df.index.isin(lab_clean.index)]
e = edges_df[edges_df["src"].isin(nodes) & edges_df["dst"].isin(nodes)]
lab = lab_clean.reindex(nodes)

G = nx.Graph(); G.add_nodes_from(nodes); G.add_edges_from(zip(e["src"],
e["dst"]))
same = lab.reindex(e["src"]).values == lab.reindex(e["dst"]).values     #
homophily

cospend = e[e["kind"] == "cospend"]
Gc = nx.Graph(); Gc.add_nodes_from(nodes);
Gc.add_edges_from(zip(cospend["src"], cospend["dst"]))
cc = list(nx.connected_components(Gc))
cid = pd.Series({a: i for i, c in enumerate(cc) for a in c}).reindex(nodes)
multi = cid.map(cid.value_counts()) > 1
purity = lab[multi].groupby(cid[multi]).nunique()                        #
99.3% equal to 1
```

B.4 Entity and random splits, features

```
from sklearn.model_selection import StratifiedGroupKFold, train_test_split

keep_cls = [c for c in range(13) if (lab == c).sum() >= 20]
nodes2 = nodes[lab.isin(keep_cls).values]
y_all, groups = lab.reindex(nodes2), cid.reindex(nodes2)
giant = groups.value_counts().idxmax()
sgkf = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
for k, (tr_e, te_e) in enumerate(sgkf.split(np.zeros(len(nodes2)),
y_all.values, groups.values)):
    if giant not in set(groups.values[te_e]):
        break
tr_r, te_r = train_test_split(np.arange(len(nodes2)), test_size=len(te_e),
stratify=y_all.values, random_state=42)
splits = {"entity": (tr_e, te_e), "random": (tr_r, te_r)}

feat_cols = [c for c in babd.columns if c not in ("account", "SW", "label")]
X = babd.drop_duplicates("account").set_index("account").loc[nodes2,
feat_cols].values.astype("float32")
y = y_all.map({c: i for i, c in enumerate(keep_cls)}).values
```

B.5 XGBoost, with and without neighbour features

```
import scipy.sparse as sp, torch
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score

def slog(a):
    return np.sign(a) * np.log1p(np.abs(a))

pos = pd.Series(np.arange(len(nodes2)), index=nodes2)
ee = e[e["src"].isin(pos.index) & e["dst"].isin(pos.index)]
s_, d_ = pos[ee["src"]].values, pos[ee["dst"]].values
ei = np.vstack([np.r_[s_, d_], np.r_[d_, s_]])                    #
undirected edge list
A = sp.csr_matrix((np.ones(ei.shape[1]), (ei[0], ei[1])), shape=(len(nodes2),
len(nodes2)))
deg_n = np.asarray(A.sum(1)).ravel()
X_aug = np.hstack([X, (A @ slog(X)) / np.maximum(deg_n, 1)[:, None], deg_n[:,
None]]).astype("float32")

for feats in (X, X_aug):
    for s, (tr, te) in splits.items():
        m = XGBClassifier(n_estimators=300, max_depth=8, learning_rate=0.1,
tree_method="hist", device="cuda", random_state=42)
        m.fit(feats[tr], y[tr]); p = m.predict(feats[te])
```

B.6 MLP and GraphSAGE

```
import torch.nn as nn, torch.nn.functional as F
from torch_geometric.nn import SAGEConv
from sklearn.preprocessing import StandardScaler

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
edge_index = torch.tensor(ei, dtype=torch.long, device=device)
y_t = torch.tensor(y, device=device)

class MLP(nn.Module):
    def __init__(self, d, h=128, k=10, p=0.2):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(d, h), nn.ReLU(), nn.Dropout(p),
nn.Linear(h, h), nn.ReLU(), nn.Dropout(p), nn.Linear(h, k))
    def forward(self, x, ei=None):
        return self.net(x)

class SAGE(nn.Module):
    def __init__(self, d, h=128, k=10, p=0.2):
        super().__init__()
        self.c1, self.c2, self.out, self.p = SAGEConv(d, h), SAGEConv(h, h),
nn.Linear(h, k), p
    def forward(self, x, ei):
        h = F.dropout(F.relu(self.c1(x, ei)), self.p, self.training)
        h = F.dropout(F.relu(self.c2(h, ei)), self.p, self.training)
        return self.out(h)

def train_eval(kind, tr, te, epochs=300, lr=5e-3, seed=42):
    sc = StandardScaler().fit(slog(X[tr]))
    xt = torch.tensor(sc.transform(slog(X)), dtype=torch.float32,
device=device)
    cnt = np.bincount(y[tr], minlength=10)
    w = (cnt.sum() / (10 * cnt)) ** 0.5
    w = torch.tensor(w / w.mean(), dtype=torch.float32, device=device)
    torch.manual_seed(seed)
    m = (SAGE(X.shape[1]) if kind == "GraphSAGE" else
MLP(X.shape[1])).to(device)
    opt = torch.optim.Adam(m.parameters(), lr=lr, weight_decay=5e-4)
    lf = nn.CrossEntropyLoss(weight=w)
    trt = torch.tensor(tr, device=device)
    for ep in range(epochs):
        m.train(); opt.zero_grad()
        loss = lf(m(xt, edge_index)[trt], y_t[trt]); loss.backward();
```

```
opt.step()
    m.eval()
    with torch.no_grad():
        return m(xt, edge_index).argmax(1).cpu().numpy()[te]
```

B.7 Venom: overlap and counterparties

```
import ast, glob, os, re

v = pd.read_csv(r"../Venom/venom-dataset-summary.csv", sep=";", decimal=",")
v = v[v["comment"].isna()].copy()
venom_addrs = set(a for l in v["btc_addrs"].fillna("
[]").apply(ast.literal_eval) for a in l)
lab_any = babd_lab.groupby("account")["label"].agg(lambda s:
tuple(sorted(set(s))))
hit = lab_any[lab_any.index.isin(venom_addrs)]                          # 2
addresses

sel = pd.read_pickle(r"../Venom/data/selected_clusters.pkl")
seed_addrs = set(a for l in sel["addr_key"] for a in l)
rows = []
for path in glob.glob(os.path.join(r"../Venom/data/raw", "c*_a*.json")):
    cid_v = int(re.search(r"c(\d+)_a", os.path.basename(path)).group(1))
    with open(path) as fh:
        for t in json.load(fh):
            for v_ in t["vin"]:
                a = (v_.get("prevout") or {}).get("scriptpubkey_address")
                if a: rows.append((cid_v, t["txid"], "input side", a))
            for o in t["vout"]:
                a = o.get("scriptpubkey_address")
                if a: rows.append((cid_v, t["txid"], "output side", a))
allrows = pd.DataFrame(rows, columns=["set", "txid", "side",
"addr"]).drop_duplicates()
allrows["is_seed"] = allrows["addr"].isin(seed_addrs)
allrows["is_exch"] = allrows["addr"].map(lab_any).apply(lambda t:
isinstance(t, tuple) and 3 in t)
g = allrows.groupby(["set", "txid"])
roles = pd.DataFrame({
    "seed_in":  g.apply(lambda d: (d["is_seed"] & (d["side"] == "input
side")).any()),
    "seed_out": g.apply(lambda d: (d["is_seed"] & (d["side"] == "output
side")).any()),
    "exch_in":  g.apply(lambda d: (d["is_exch"] & (d["side"] == "input
side")).any()),
    "exch_out": g.apply(lambda d: (d["is_exch"] & (d["side"] == "output
side")).any()),
```

```
}).reset_index()
r25 = roles[roles["set"] == 25]
print(int((r25.seed_in & r25.exch_in).sum()), int((r25.seed_out &
r25.exch_out).sum()))   # 119 237
```